Metaheuristics: Computer Decision-Making, pp. 465-480
Daniel Merkle and Martin Middendorf

22 ON THE BEHAVIOR OF ACO ALGORITHMS: STUDIES ON SIMPLE PROBLEMS

Daniel Merkle[1] and Martin Middendorf[1]

[1]Department of Computer Science
University of Leipzig
D-04109 Leipzig, Germany
merkle@informatik.uni-leipzig.de, middendorf@informatik.uni-leipzig.de

Abstract: The behavior of Ant Colony Optimization (ACO) algorithms is studied on simple problems which allow us to identify characteristic properties of these algorithms. In particular, ACO algorithms using different pheromone evaluation methods are investigated. A new method for the use of pheromone information by artificial ants is proposed. Experimentally it is shown that an ACO algorithm using the new method performs better than ACO algorithms using other known methods for certain types of problems.
Keywords: Ant colony optimization, Pheromone information, Permutation problems.

22.1 INTRODUCTION

The Ant Colony Optimization (ACO) metaheuristic has been applied successfully to several combinatorial optimization problems (for an overview see Dorigo and Di Caro (1999)). Usually ACO algorithms have been tested on benchmark problems or real world problems. In this paper, we start studying the behavior of ACO algorithms on simple problems. The aim of this study is to show that several interesting properties of ACO algorithms can be observed much more clearly when applying them to quite simple problems instead of large irregular problems. Therefore, our observations help to understand the behavior of ACO algorithms also on more complex problems.

ACO algorithms are based on the principle of communication by pheromones that has been observed by real ants. In ACO algorithms, artificial ants

that found a good solution mark their paths through the decision space by putting some amount of pheromone along the path. Alternatively, all ants are allowed to mark their paths but the better the solution of an ant is the more pheromone it uses. The following ants of the next generation are attracted by the pheromone so that they will search in the solution space near good solutions. A characteristic feature of the original ACO metaheuristic is that the ants make their decisions based on local pheromone information that correspond directly to the actual decision (Dorigo et al., 1999). It was shown in Merkle and Middendorf (2000) that ACO algorithms with a global use of the pheromone information can improve the optimization behavior for some types of problems. In this paper, we propose a new global pheromone evaluation method, called relative pheromone evaluation. It is shown experimentally that relative pheromone evaluation performs better than the standard local evaluation method and the global summation evaluation method on some of our test problems. We argue that relative pheromone evaluation will be interesting also for solving more complex problems.

The paper is organized as follows. All test problems are permutation problems which are introduced in the next Section 22.2. In Section 22.3 we describe the ACO algorithm with the different pheromone evaluation methods. The choice of the parameter values of the algorithms that are used in the test runs and the test instances are described in Section 22.4. Results are reported in Section 22.5 and a conclusion is given in Section 22.6.

22.2 PROBLEMS

We consider the following type of permutation problems which is known as the Linear Assignment problem. Given are n items $1, 2, \ldots, n$ and an $n \times n$ cost matrix $C = [c(i,j)]$ with $c(i,j) \geq 0$. Let $\mathcal{P}_n$ be the set of permutations of $(1, 2, \ldots, n)$. For a permutation $\pi \in \mathcal{P}_n$ let $c(\pi) = \sum_{i=1}^{n} c(i, \pi(i))$ be the cost of the permutation. The problem is to find a permutation $\pi \in \mathcal{P}_n$ of the items that has minimal costs, i.e., $c(\pi) = \min\{c(\pi') \mid \pi' \in \mathcal{P}_n\}$. It should be noted, that the Linear Assignment problem is not an NP-hard problem and can be solved in polynomial time (Papadimitriou and Steiglitz, 1982).

22.3 THE ACO ALGORITHMS

The ACO algorithm for permutation problems consists of several iterations where in every iteration each of m ants constructs one permutation. Every ant selects the items for the places of the permutation in some order. For the selection of an item the ant uses pheromone information which stems from former ants that have found good solutions. In addition an ant may also use heuristic information for its decisions. The pheromone information, denoted by τ_{ij}, and the heuristic information, denoted by η_{ij}, are indicators of how good it seems to have item j at place i of the permutation.

The next item is chosen from the set S of items that have not been placed so far according to a probability distribution. In this paper, we use three different

methods to determine the probabilities. The first method is the standard *local evaluation* (L-Eval) of the pheromone values that is used in nearly all ACO algorithms in the literature (e.g. Dorigo et al. (1996)). According to this method, the probability p_{ij} to select item $j \in \mathcal{S}$ on place i depends only on the local pheromone values and heuristic values in the same row, i.e., τ_{il}, η_{il} $l \in [1:n]$, and is determined by

$$p_{ij} = \frac{\tau_{ij} \cdot \eta_{ij}}{\sum_{h \in \mathcal{S}} \tau_{ih} \cdot \eta_{ih}}. \tag{22.1}$$

The second method is a form of global pheromone evaluation where p_{ij} can depend on the pheromone τ_{kl} or heuristic values η_{kl} with $k \neq i$. This method is called *summation evaluation* (S-Eval). The use of global pheromone evaluation and the summation evaluation have been proposed by Merkle and Middendorf (2000). For summation evaluation an ant considers for deciding which item to put on place i the sums of all pheromone values in the columns up to the row i. Summation evaluation is attractive for problems like tardiness problems in machine scheduling where there is only a small interval of places in the schedule which are good for a job. The probability p_{ij} to select item $j \in \mathcal{S}$ on place i is determined by

$$p_{ij} = \frac{(\sum_{k=1}^{i} \tau_{kj}) \cdot \eta_{ij}}{\sum_{h \in \mathcal{S}} (\sum_{k=1}^{i} \tau_{kh}) \cdot \eta_{ih}} \tag{22.2}$$

According to this method, an item j that has high pheromone values τ_{kj} for $k < i$, but has yet not been scheduled, is likely to be scheduled soon. This property is desirable for tardiness problems since it is important that jobs are scheduled not too late (before their deadline). Summation evaluation has been shown to perform well for the Single Machine Weighted Total Tardiness problem (Merkle and Middendorf, 2000), the Job Shop problem (Teich et al., 2001), the Flow-Shop problem (Rajendran and Ziegler, 2003), and the Assembly Line problem (Bautista and Pereira, 2002). A combination of local evaluation and summation evaluation performed best for the Resource-Constrained Project Scheduling problem (Merkle et al., 2002).

The third rule is a newly proposed pheromone evaluation rule that we call *relative (pheromone) evaluation* (R-Eval). Relative evaluation is motivated by the following observation: Assume that an item j has not been placed by an ant on a place $< i$ although a large part of the pheromone in column j lies on the first $i-1$ elements $\tau_{1j}, \ldots, \tau_{(i-1)j}$ in the column. In this case, all values τ_{kj} for $k \geq i$ are small. Hence, it is unlikely that the ant will place item j on place i when some other item has a high pheromone value for place i. This is true even when τ_{ij} is the highest of all pheromone values $\tau_{ij}, \ldots, \tau_{nj}$ on the remaining places. However, for many optimization problems it might still be important on which place $l \geq i$ item j is, even when these places are not the most favorite ones. For such problems (i.e. when there are several good places for an item) we propose not to use directly the pheromone values in each row but to normalize them with the relative amount of pheromone in the rest of

their columns. Hence, the probability p_{ij} to select item $j \in \mathcal{S}$ on place i is determined by

procedure ANTALGORITHM(X,y)
parameter $X \in \{L, S, R\}$, $y \in \{f, b, p\}$);
repeat
  foreach ant $k \in \{1, \ldots, m\}$
    if $y = p$
      Permute the rows of the pheromone and cost matrix according to the given permutation π_k;
    if $y = b$
      Reverse the order of the rows of the pheromone and cost matrix;
    $S = \{1, 2, \ldots, n\}$; *set of available items*
    for $i = 1$ **to** n
      Choose item $j \in \mathcal{S}$ with probability p_{ij};
      $\mathcal{S} := \mathcal{S} - \{j\}$;
    endfor
    Undo the permutation of the pheromone and cost matrix;
  endfor
  foreach (i, j)
    $\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij}$; *evaporate pheromone*
  endfor
  foreach $(i, j) \in$ best solution
    $\tau_{ij} \leftarrow \tau_{ij} + \frac{1}{\text{cost of best solution}+1}$; *update pheromone*
  endfor
until stopping criterion is met

Algorithm 22.1 Ant algorithm.

$$p_{ij} = \frac{\tau^*_{ij} \cdot \eta_{ij}}{\sum_{h \in \mathcal{S}} \tau^*_{ih} \cdot \eta_{ih}} \quad \text{with} \quad \tau^*_{ij} := \left(\frac{\sum_{k=1}^{n} \tau_{kj}}{\sum_{k=i}^{n} \tau_{kj}} \right)^{\gamma} \cdot \tau_{ij} \tag{22.3}$$

where $0 \leq \gamma \leq 1$ is a parameter that allows to determine the strength of normalization.

In those cases where a heuristic was used, the values η_{ij} were computed according the following formula $\eta_{ij} = 1/c(i, j)$.

The best solution found by an ant in the current iteration is then used to update the pheromone matrix by adding some amount of pheromone for every element corresponding to the solution. But before the update is done a percentage of the old pheromone is evaporated according to the standard formula $\tau_{ij} = (1 - \rho) \cdot \tau_{ij}$. Parameter ρ allows to determine how strongly old

pheromone influences the future. Then, for every item j of the best permutation found so far some amount of pheromone Δ is added to element τ_{ij} of the pheromone matrix where i is the place of item j in the solution. The algorithm stops when some stopping criterion is met, e.g. a certain number of generations has been done.

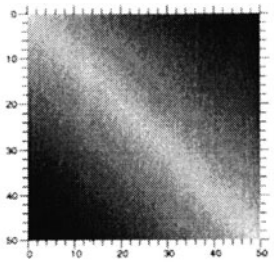
We extend the ACO algorithm described above by possibly changing the sequence in which the ants assign items to the places. For each ant k in a generation a permutation π_k is given that is used by the ant to permute the rows of the pheromone matrix and the cost matrix before the ant starts constructing a solution. We consider the following cases i) no permutation of the rows (forward ants), ii) reversing the sequence of rows in the matrices (backward ants), and iii) performing a random permutation before an ant starts (permutation ants). The advantage of using forward and backward ants for the Shortest Common Supersequence problem was shown by Michels and Middendorf (1999). Merkle and Middendorf (2001) showed that using different random permutations for the ants can lead to an improved optimization behavior for ACO algorithms solving permutation scheduling problems because then the probabilities used by the ants to select the items usually better reflect the relative size of the corresponding products of pheromone value with heuristic value. Note, that when using summation evaluation it is not reasonable to use random permutations for the ants.

The different combinations of matrix permutation and pheromone evaluation method are denoted by X-y-Eval with $X \in \{L, S, R\}$ and $y \in \{f, b, p\}$ where f=forward, b=backward, and p=permutation.

22.4 TESTS

All results in the following section are averages over 25 runs, every run was stopped when the quality of the best found solution has not changed for 500 iterations or after 10000 generations. All test problems have $n = 50$ items. Evaporation rate was set to $\rho = 0.01$ and in some cases we also tested $\rho = 0.005$ in order to make sure that our conclusions do not depend too much on the actual value of ρ. Pheromone update was done with $\tau_{ij} = \tau_{ij} + 1/(\text{cost of solution}+1)$ and the pheromone matrix was initialized with $\tau_{ij} = 1/(\rho \cdot c(\pi_{opt}))$ (this is also the maximal possible value of an element in the pheromone matrix) where $c(\pi_{opt})$ is the quality of an optimal solution, $i, j \in [1, 50]$. The tests have been done with and without using the heuristic, i.e., $\eta_{ij} = 1$ for $i, j \in [1, 50]$. The results with heuristic are similar to the results without heuristic only the convergence is faster. Therefore we describe only the results obtained without using the heuristic in the next section. Note also, that X-b-Eval with $X \in \{L, S, R\}$ is tested only for asymmetric instances were an interesting difference between the results for X-b-Eval and X-f-Eval might occur.

Table 22.1 Cost matrix and results for an instance of type1: brighter gray pixel indicate smaller costs; given are average quality of best solution (Best), average iteration number when best solution was found (Gen.).



Evaluation		Type 1	
		Best	Gen.
L-Eval	f	287.4	3456
	p	50.4	3547
R-Eval	f	70.1	2421
	p	50.1	3415
S-Eval	f	51.9	2932

22.5 RESULTS

Problem instances of type 1 reflect the situation that there exists only a single optimal solution. Without loss of generality we assume that the optimal solution is to place item i at place i for $i \in [1:n]$. We consider the case where the costs for placing an item grow the more its actual place differs from its optimal place. The cost matrix is rather simple (see Table 22.1). For each element (i,j) of the cost matrix the cost $c(i,j)$ equals the distance from the main diagonal, i.e., $c(i,j) = |i-j|+1$. Clearly, to place item i, $i = 1,\ldots,n$ on place i is the only optimal solution with costs n.

Table 22.1 shows the results of different X-y-Eval ACO algorithms on the problem instance of size 50. L-p-Eval, R-p-Eval, and S-f-Eval are the best methods while L-f-Eval is worst. This confirms that summation evaluation is a good method for problems where there exists (more or less) a single good place for every item. Both other methods perform worse when using forward ants. Figure 22.1 shows that L-f-Eval tends to choose items too early (in the upper 2/3rds of the pheromone matrix the values above the diagonal are higher than below). Note that this effect has been observed before by Merkle and Middendorf (2001). It can also be seen that L-f-Eval converges earlier in the uppermost rows of the matrix than in the more lower rows. It is interesting that the new proposed relative evaluation method performs much better when using forward ants than L-Eval. Compared to L-f-Eval the results of all other methods depicted in Figure 22.1 show a very symmetric behavior and do not have the undesired bias. Local evaluation and relative evaluation profit clearly from using permutation ants for this type of problem.

Problem instances of type 2 are similar to problem instances of type 1 but the places of some of the items are not relevant now. Hence there exist several optimal solutions which differ only for some of the items. For the test instances the elements in the first $k < n$ rows of the cost matrix are one and the other rows of the cost matrix are the same as for the cost matrix of in-

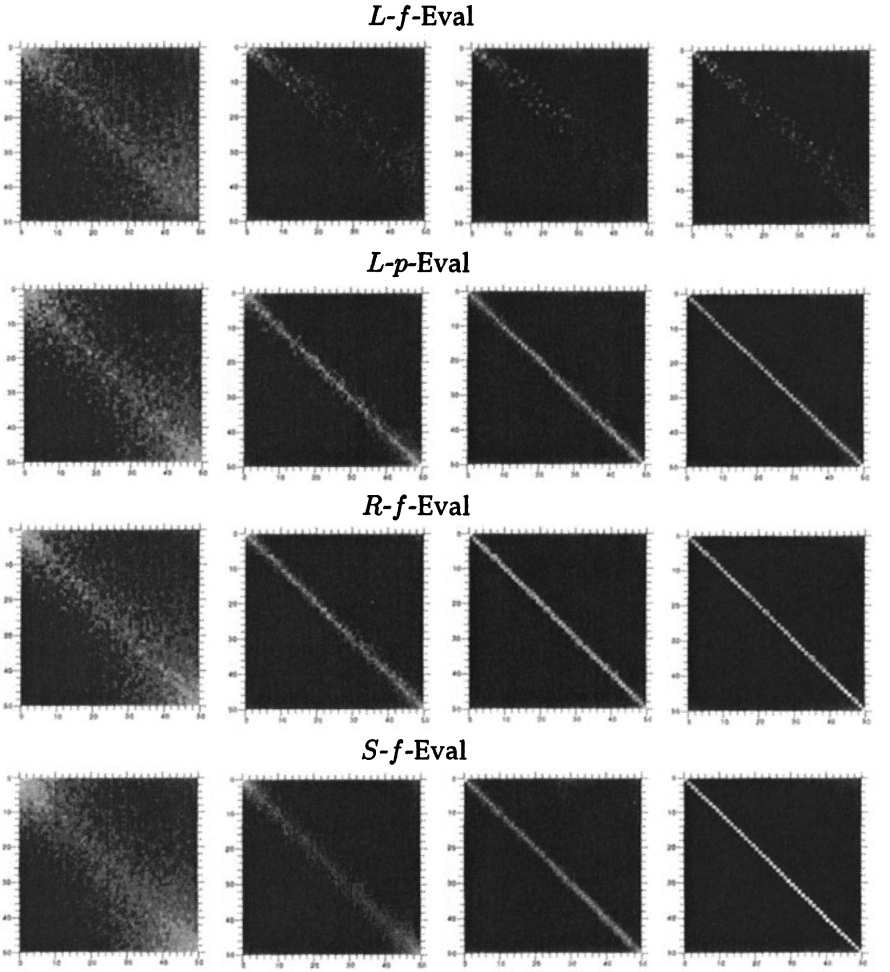
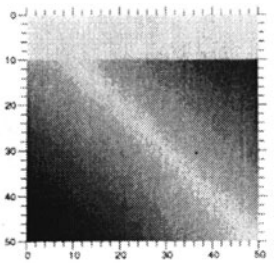


Figure 22.1 Instance of type 1: Average pheromone matrices in iterations 800, 1500, 2200, and 2900: brighter gray colors indicate higher pheromone values.

stance of type 1 (see Table 22.2). Hence, every optimal solution has item i on place i for $i > k$ and the places for items $1, \ldots, k$ are not relevant. The results for $n = 50$ and $k = 10$ that are presented in Table 22.2 show that similar as for problem instance 1 very good results are found by L-p-Eval and R-p-Eval. Also S-b-Eval found very good results. It can be seen that there is a significant difference between the quality obtained by backward or forward ants. In all cases the results of the backwards ants are better. Figure 22.2 shows that the high pheromone values for L-b-Eval are much more concentrated along the diagonal than for L-f-Eval. This indicates that it is better for when the important decisions can be made early. This observation might explain why den Besten et al. (1999) found that their ant algorithm for the single machine total tardiness problem had more difficulties with instances where most jobs have their due date late. Compared to L-Eval and S-Eval the relative difference between the results for forward and backward ants with R-Eval are smaller.

Table 22.2 Cost matrix and results for an instance of type 2: brighter gray pixel indicate smaller costs; given are average quality of best solution (Best), average iteration number when best solution was found (Gen.).



Evaluation		Type 2	
		Best	Gen.
L-Eval	f	270.5	3490
	b	207.9	3183
	p	50.0	3308
R-Eval	f	62.3	2256
	b	70.2	2345
	p	51.3	3557
S-Eval	f	254.2	2791
	b	50.6	2604

For problem instances of type 3 there exist two optimal solutions — called base solutions — which are disjoint in the sense that the sets of corresponding decisions are disjoint. For some instances there exist additional optimal solutions that are obtained by a combination of the decisions corresponding to the base solutions. Again, every element on the main diagonal of the cost matrix is one. Moreover, all elements on another line (called 1-line) that is parallel to the main diagonal are one, i.e., all elements (i, j) with $j - i = k \bmod n$ for a fixed $k \in [1, n]$ are one. All other elements of the cost matrix are two. The instances of type 3 differ only in the distance k of the 1-line from the diagonal. For $n = 50$ we tested the algorithms on matrices with values $k \in \{3, 23, 25\}$.

L-Eval and R-Eval perform quite well on instances of type 3 and mostly the algorithm found optimal solutions (see Table 22.3). S-Eval does not perform very well since for each item there are two good places which are not

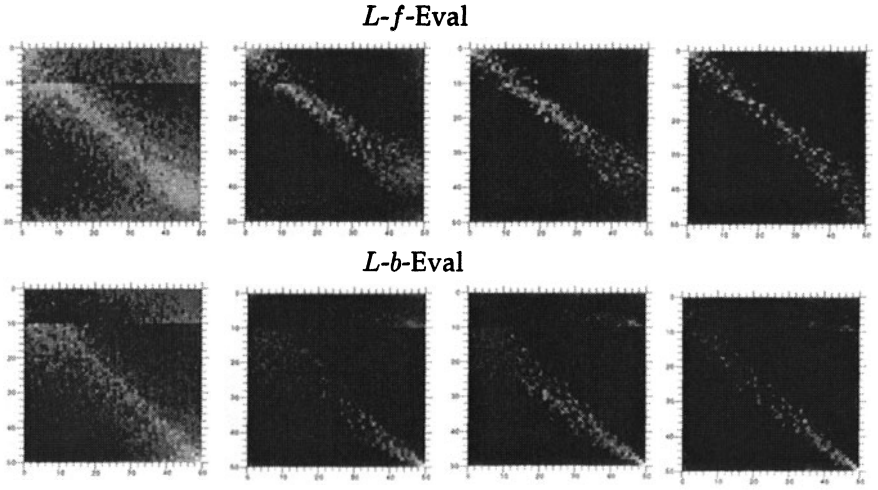


Figure 22.2 Instance of type 2: Average pheromone matrices in iterations 800, 1500, 2200, and 2900: brighter gray colors indicate higher pheromone values.

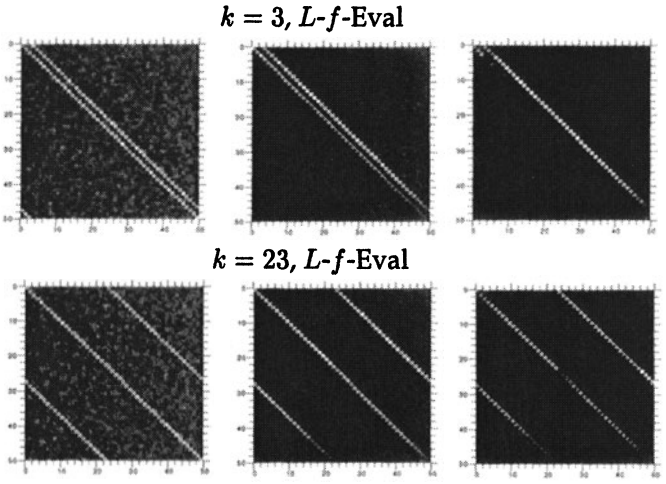
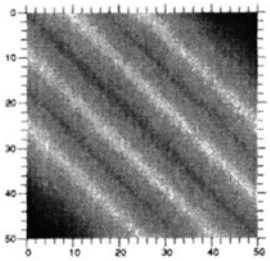


Figure 22.3 Instances of type 3 with $k = 3$ and $k = 23$: Average pheromone matrices in iterations 100, 800, 1500: brighter grey colors indicate higher pheromone values.

Table 22.3 Results on instances of type 3: average quality of best solution (Best), average number of iteration when best solution was found (Gen.), and average relation of number of items placed according to diagonal and according to 1-line in the best found solution (D/L).

Evaluation		$k = 3$			k=23			k=25		
		Best	Gen.	D/L	Best	Gen.	D/L	Best	Gen.	D/L
L-Eval	f	51.2	1770	0.03	53.3	1919	0.93	50	1538	0.89
	p	52.0	3011	1.01	52.1	3012	0.93	50	1850	0.95
R-Eval	f	50.9	1653	0.06	52.2	2009	0.95	50	1500	0.93
	p	52.0	3118	1.01	51.8	3094	1.11	50	1872	0.98
S-Eval	f	72.5	6193	4.13	72.4	6502	1.15	72.6	6504	1.01

Table 22.4 Cost matrix and results for an instance of type 4: brighter gray pixel indicate smaller costs; given are average quality of best solution (Best), average iteration number when best solution was found (Gen.), results with $\rho = 0.005$ are in brackets.



Evaluation		Type 4	
		Best	Gen.
L-Eval	f	109.0 (105.4)	3200 (8977)
	p	50.0	4493
R-Eval	f	55.4 (50.8)	2657 (5164)
	p	50.2	3941
S-Eval	f	194.4 (192.7)	1993 (2970)

neighbored. It is interesting that the instance with $k = 25$ could always be solved optimal when using L-Eval and R-Eval. The reason is that instances with $k = 3$ and $k = 23$ have only two different optimal solutions: i) the diagonal with item $i \in [1, 50]$ on place i ii) the 1-line with item $i \in [1, 50]$ on place $i - k \bmod 50$ where k is the distance between the diagonal and the 1-line. For the instance with $k = 25$ there exist 2^{25} optimal solutions: for $j \in [1, 25]$ items j and $j + 25$ are placed either on places j and $j + 25$ or on places $j - k \bmod 50$ and $j + 25 - k \bmod 50$. Another interesting observation is that L-f-Eval and R-f-Eval prefer to place elements according to the 1-line for $k = 3$. But this is not the case for $k = 23$ (see Figure 22.3). The reason is that choosing item 4 for place 1 will not allow to place item 4 on place 4. Hence it is likely that item 7 is placed on place 4. In other words choosing an item according to the 1-line in the first generations enforces that another item is placed according to the 1-line k decision later. So for small k this effect is of importance when using forward ants.

Similar as for instances of type 3 there exist several optimal solutions for instances of type 4. But for instances of type 4 it is more difficult to find these optimal solutions. The matrix has four diagonal lines with elements of cost one. For all other elements in the matrix the cost is equal to the minimal distance from the next diagonal line plus one (see Table 22.4). L-p-Eval and R-Eval performed best and find always or nearly always an optimal solution while — as expected for this type of problems — S-Eval performs much worse even for $\rho = 0.005$ (see Table 22.4). It is interesting that L-f-Eval performed much worse than L-p-Eval which performed best (even for $\rho = 0.005$ the results for L-f-Eval do not improve much). One reason is that L-f-Eval tends to converge early in the upper part of the matrix which is not the case for L-p-Eval (see Figure 22.4). For this type of problem starting to converge in the upper rows early is dangerous because there exist many solutions that have small costs in the upper rows but which can not be extended to an optimal or nearly optimal solution. R-Eval performs very well, no matter whether forward or

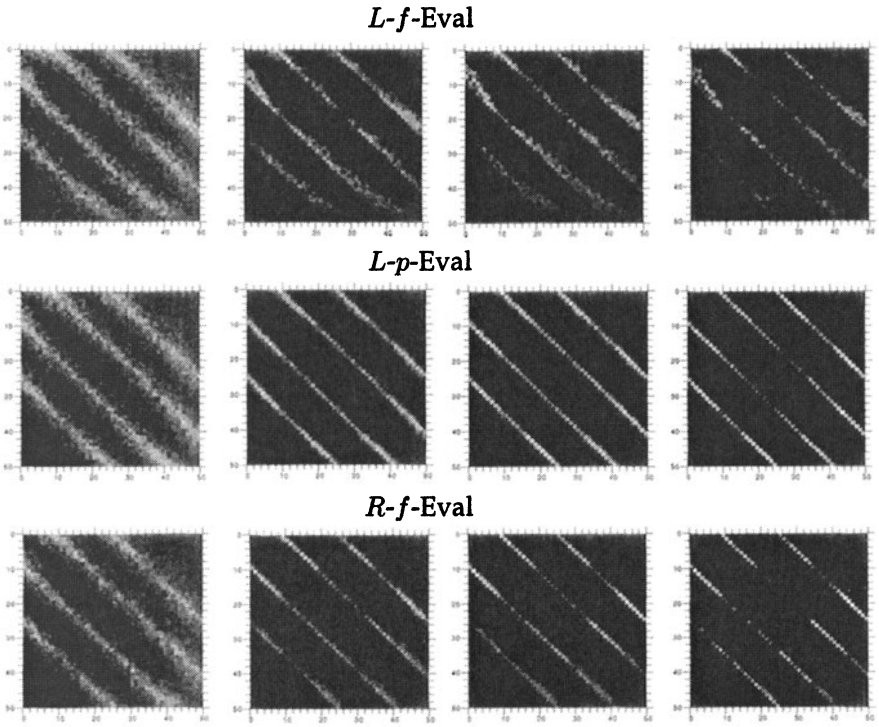


Figure 22.4 Instance of type 4: Average pheromone matrices in iterations 800, 1500, 2200, and 2900: brighter gray colors indicate higher pheromone values.

permutation ants are used. R-f-Eval does not show early convergence in the upper rows (see Figure 22.4) since it has the property that pheromone differences in lower rows have an influence even for items which have some high pheromone values in some of the upper rows. Thus when in early phases of a run an item has been placed mostly onto one of the first places this decision has good chances to be revised when later during a run a few ants found that it is better to place this item late.

22.6 CONCLUSION

We have shown that simple permutation problems are well suited to identify and explain some characteristic properties of Ant Colony Optimization (ACO) algorithms. In particular, we identified characteristic properties of different pheromone evaluation methods. A new method to use the pheromone information (relative evaluation) was proposed and compared to known standard methods. Experiments on the simple test problems show that the new method performs best on a certain type of problems for which it was designed. It is interesting to study whether relative evaluation is also a good method for harder and real world problems.

22.6.1 A final note

Independently Stützle and Dorigo (2001) studied ACO algorithms on simple problems. In particular, they investigate the influence of pheromone information and evaporation on the convergence on simple graph problems.

J. Bautista and J. Pereira. Ant algorithms for assembly line balancing. In M. Dorigo, G. Di Caro, and M. Sampels, editors, *Ant algorithms*, number 2463 in Lecture Notes in Computer Science (LNCS), pages 65–75, Berlin, 2002. Springer.

M. den Besten, T. Stützle, and M. Dorigo. Scheduling single machines by ants. Technical Report IRIDIA/99-16, IRIDIA, Université Libre de Bruxelles, Belgien, 1999.

M. Dorigo, G. Di Caro, and L. M. Gambardella. Ant algorithms for discrete optimization. *Artificial Life*, 5(2):137–172, 1999.

M. Dorigo and G. Di Caro. The ant colony optimization meta-heuristic. In D. Corne, M. Dorigo, and F. Glover, editors, *New ideas in optimization*, pages 11–32. McGraw-Hill, London, 1999.

M. Dorigo, V. Maniezzo, and A. Colorni. The ant system: optimization by a colony of cooperating agents. *IEEE Trans. Systems, Man, and Cybernetics – Part B*, 26:29–41, 1996.

D. Merkle and M. Middendorf. An ant algorithm with a new pheromone evaluation rule for total tardiness problems. In S. Cagnoni, R. Poli, Y. Li, G. Smith, D. Corne, M.J. Oates, E. Hart, P.L. Lanzi, E.J.W. Boers, B. Paechter, and T.C. Fogarty, editors, *Real-World Applications of Evolutionary Computing, Proceedings of the EvoWorkshops 2000*, number 1803 in Lecture Notes in Computer Science (LNCS), pages 287–296, Berlin, 2000. Springer.

D. Merkle and M. Middendorf. A new approach to solve permutation scheduling problems with ant colony optimization. In E.J.W. Boers, J. Gottlieb, P.L. Lanzi, R.E. Smith, S. Cagnoni, E.Hart, G.R. Raidl, and H. Tijink, editors, *Applications of Evolutionary Computing, Proceedings of the EvoWorkshops 2001*, number 2037 in Lecture Notes in Computer Science (LNCS), pages 484–493. Springer Verlag, 2001.

D. Merkle, M. Middendorf, and H. Schmeck. Ant colony optimization for resource-constrained project scheduling. *IEEE Transactions on Evolutionary Computation*, 6(4):333–346, 2002.

R. Michels and M. Middendorf. An ant system for the shortest common supersequence problem. In D. Corne, M. Dorigo, and F. Glover, editors, *New Ideas in Optimization*, pages 51–61. McGraw-Hill, London, 1999.

C.H. Papadimitriou and K. Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, 1982.

C. Rajendran and H. Ziegler. Ant-colony algorithms for permutation flowshop scheduling to minimize makespan / total flowtime of jobs. *European Journal of Operational Research*, 2003. To appear.

T. Stützle and M. Dorigo. An experimental study of the simple ant colony optimization algorithm. In N. Mastorakis, editor, *Proceedings of the 2001 WSES International Conference on Evolutionary Computation (EC'01)*, pages 253–258. WSES-Press International, 2001.

T. Teich, M. Fischer, A. Vogel, and J. Fischer. A new ant colony algorithm for the job shop scheduling problem. In L. Spector, D. Whitley, D. Goldberg, E. Cantu-Paz, I. Parmee, and H.-G. Beyer, editors, *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2001)*, page 803, San Francisco, CA, USA, 2001. Morgan Kaufmann.